Final Project

Design a Long-Running Agent Workflow for a Finance Task

International School of Finance, Fudan University Hulai Zhang

Individual project. Choose a recurring finance task and design an agent-based workflow around it. It should retain state, handle new inputs, and resume after interruption—not be a single prompt or a general-purpose autonomous analyst.

1. Specify your task

Select a task you perform or can realistically describe: for example, **buy-side** signal research, strategy review, or portfolio-risk monitoring; **sell-side** earnings updates, company research, or valuation refreshes. Other finance tasks are welcome. Define the intended user, manual baseline, trigger or frequency, inputs and permitted data sources, information cutoff, expected work product, and concrete acceptance criteria. Keep the scope manageable.

2. Design the architecture using the lecture concepts

Provide an annotated workflow diagram and a short design note explaining:

- **Roles and handoffs:** agent responsibilities, tools, reusable skills or instructions, and input/output artifacts. Justify the division of work; use deterministic code for calculations where appropriate. Distinguish agent decisions from the harness/controller that manages execution.
- **Long-running operation:** persistent state, checkpoints, scheduling or new-input triggers, dependencies, retries, and stop/escalation rules. Explain how an interrupted run resumes without repeating completed actions, and set limits on time, cost, and tool permissions.
- **Verification and improvement:** checks independent of the producing agent, human approval before consequential actions, and a bounded revision loop. Explain how a proposed change is tested against a fixed baseline, accepted or rejected, versioned, and rolled back. For time-sensitive tasks, prevent future-information leakage.

3. Make the design inspectable

Include one end-to-end example from input to work product, with intermediate artifacts and acceptance checks, plus one failure/interruption-and-recovery example. A small runnable prototype is welcome; a clearly labelled, step-by-step design walkthrough is acceptable. Distinguish executed results from simulated examples and identify limitations. Use public, synthetic, or properly licensed inputs; keep live trading, client publication, and other consequential actions simulated or behind explicit human approval.

4. Prepare the repository

Include your design note and diagram, prompts/skills/configuration, code if implemented, shareable example inputs and outputs, and verification/recovery evidence. The root `README.md` must give your identification, task summary, **repository URL**, file guide, and setup/run or walkthrough instructions (including dependencies where relevant). Disclose AI assistance and explain what you checked yourself. Use meaningful commits throughout development; do not submit only a final-file dump.

5. Submit one ZIP with complete Git history

GitHub, GitLab, or an equivalent Git host is acceptable. Commit and push the submission files and relevant development branches/tags, then archive a **full, non-shallow local clone, including its hidden `.git/` directory**. For example, on macOS/Linux, run from the parent folder:

```
git clone <repository-url> final-project
zip -r studentID-final-project.zip final-project
```

A host's "Download ZIP" or `git archive` is not sufficient: neither includes Git history. Extract your ZIP into a new folder and check that the README, all deliverables, and `.git/` are present; `git log --all --oneline` must show the development history and `git fsck --full` must pass. The archive must be self-contained for reviewing files and history, without relying on repository access. Never include credentials, private client data, or secrets in files *or commit history*.